

Conectar OpenClaw con Ollama en Windows sin Lynkr

La solución más rápida y fiable es usar el comando integrado `ollama launch openclaw`, que **autoconfigura todo en segundos**. Si eso falla, la configuración manual del archivo `openclaw.json` resuelve el error "Unknown model" en menos de 5 minutos apuntando directamente al endpoint OpenAI-compatible de Ollama (<http://127.0.0.1:11434/v1>). Lynkr no es necesario: Ollama ya expone nativamente una API compatible con OpenAI, y OpenClaw la soporta directamente. Para un setup CPU-only con 32GB RAM, el modelo `gpt-oss:20b` es la mejor opción gracias a su arquitectura MoE que activa solo **3.6B parámetros por token** de sus 21B totales.

Solución 1: el comando oficial que lo resuelve todo

Ollama incluye integración oficial con OpenClaw desde enero de 2026. Un solo comando configura la conexión completa:

```
ollama launch openclaw
```

Este comando hace tres cosas automáticamente: detecta los modelos instalados, genera la configuración necesaria en `~/.openclaw/openclaw.json`, y reinicia el gateway de OpenClaw. Si el gateway ya está corriendo, lo recarga con la nueva configuración. El alias antiguo `ollama clawdbot` también funciona. Si solo quieres generar la configuración sin lanzar nada, usa `ollama launch openclaw --config`.

Tiempo estimado: 30 segundos. Si esto no funciona o necesitas control más granular, pasa a la Solución 2.

Solución 2: configuración manual de `openclaw.json`

El error "Unknown model" ocurre porque OpenClaw prefixa internamente los nombres de modelo con `ollama/` y, si la configuración no registra explícitamente los modelos, el sistema no sabe cómo enrutarlos. La solución es declarar el proveedor Ollama manualmente en el archivo de configuración.

Paso 1: Abre el archivo de configuración (formato JSON5, admite comentarios):

```
%USERPROFILE%\openclaw\openclaw.json
```

Paso 2: Añade o modifica la sección `models.providers` con esta estructura:

```
{
  "models": {
    "providers": {
      "ollama": {
```

```

"baseUrl": "http://127.0.0.1:11434/v1",
"apiKey": "ollama",
"api": "openai-responses",
"models": [
  {
    "id": "gpt-oss:20b",
    "name": "GPT-OSS 20B",
    "reasoning": true,
    "contextWindow": 8192,
    "maxTokens": 4096
  },
  {
    "id": "mistral:latest",
    "name": "Mistral 7B",
    "contextWindow": 8192,
    "maxTokens": 4096
  },
  {
    "id": "llama2:latest",
    "name": "Llama 2",
    "contextWindow": 4096,
    "maxTokens": 2048
  },
  {
    "id": "qwen3:latest",
    "name": "Qwen 3",
    "contextWindow": 8192,
    "maxTokens": 4096
  }
]
}
},
"agents": {
  "defaults": {
    "model": {
      "primary": "ollama/gpt-oss:20b",
      "fallbacks": ["ollama/mistral:latest"]
    }
  }
}
}
}

```

Paso 3: Verifica que los IDs de modelo coincidan exactamente con lo que reporta Ollama:

```
ollama list
```

Los nombres deben coincidir carácter por carácter, incluyendo tags (: latest, : 20b, etc.). **Esta discrepancia en los nombres es la causa más común del error "Unknown model".**

Paso 4: Configura la variable de entorno para auto-descubrimiento (opcional pero recomendado):

```
setx OLLAMA_API_KEY "ollama-local"
```

Esto permite que OpenClaw descubra automáticamente modelos adicionales vía /api/tags.

Paso 5: Reinicia el gateway de OpenClaw:

```
openclaw gateway restart
```

Paso 6: Verifica la salud del sistema:

```
openclaw doctor
```

Tiempo estimado: 5-10 minutos. Este método da control total sobre qué modelos están disponibles y cómo se configuran.

Nota crítica sobre `contextWindow` en CPU

Con 32GB RAM y sin GPU, mantén `contextWindow` entre **4096 y 8192 tokens**. Valores mayores disparan el uso de memoria del KV cache y ralentizan drásticamente la inferencia. Aunque `gpt-oss:20b` soporta hasta 128K tokens de contexto, en CPU puro un contexto grande puede hacer que cada respuesta tarde minutos.

Por qué Lynkr no es necesario (y cómo evitarlo)

Lynkr es un proxy que traduce llamadas API entre formatos (Anthropic ↔ OpenAI ↔ Ollama), pero **OpenClaw ya habla directamente con el endpoint OpenAI-compatible de Ollama**. El proxy es redundante para este caso. Los problemas de compilación en Windows son reales: Lynkr depende de `tree-sitter` y `better-sqlite3`, ambos módulos nativos C++ que requieren compilación vía `node-gyp`, y `tree-sitter` tiene bugs documentados de compilación en Windows 10/11.

Si por algún motivo necesitaras Lynkr en el futuro, hay tres formas de evitar los errores de compilación:

- **Docker:** `docker-compose up -d` en el repositorio de Lynkr — esquiva completamente el problema de compilación nativa
- **WSL2:** Instalar Lynkr dentro de Windows Subsystem for Linux, donde las dependencias C++ compilan sin problemas
- **Build Tools completo:** Instalar Visual Studio Build Tools con el workload "Desktop development with C++" (incluye MSVC, Windows SDK y CMake), luego `npm install -g lynkr`

Pero insisto: **para conectar OpenClaw a Ollama, Lynkr no aporta nada que la conexión directa no resuelva.**

Alternativas si la conexión directa falla

Si la configuración directa de OpenClaw presenta problemas (el proyecto es conocido por ser inestable y "crashear mucho", según su propia comunidad), estas son las alternativas ordenadas por practicidad:

LiteLLM es la alternativa más recomendable. Es un proxy Python puro sin dependencias nativas de C+, se instala con `pip install litellm[proxy]` y soporta más de 100 proveedores LLM. Para Ollama:

```
pip install litellm[proxy]
litellm --model ollama/gpt-oss:20b --port 4000
```

Esto crea un endpoint OpenAI-compatible en `http://localhost:4000/v1` que OpenClaw puede consumir configurando un proveedor custom con esa URL. **Tiempo de instalación: 2 minutos.** Funciona en Windows sin problemas de compilación.

LM Studio ofrece una GUI nativa de Windows con servidor OpenAI-compatible integrado. Descarga modelos con un clic y expone un endpoint en `http://localhost:1234/v1`. Es la opción más visual, aunque duplica funcionalidad con Ollama. Útil si prefieres interfaz gráfica.

Docker Desktop funciona bien en Windows 10/11 con WSL2 habilitado. Para correr Ollama en Docker:

```
docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
docker exec -it ollama ollama pull gpt-oss:20b
```

Sin embargo, **si ya tienes Ollama instalado nativamente y funcionando, Docker no mejora nada.** Docker es útil para aislar Lynkr o LiteLLM, no para Ollama en sí.

Optimización para CPU puro con 32GB RAM

Tu setup de 32GB sin GPU puede ejecutar modelos de hasta **33B parámetros** según las guías oficiales de Ollama. Aquí está el rendimiento esperado de tus modelos:

Modelo	Tamaño real en RAM	Params activos	Velocidad CPU estimada	Viabilidad
gpt-oss:20b	~14GB	3.6B (MoE)	5-8 tok/s	☑ Recomendado
mistral:7b	~4.1GB	7B	8-12 tok/s	☑ Rápido
llama2:7b	~3.8GB	7B	8-12 tok/s	☑ Rápido
qwen3 (varía por tamaño)	4-20GB	Varía	4-12 tok/s	☑ Depende del tamaño

gpt - oss : 20b es la mejor elección para tu hardware. Su arquitectura Mixture of Experts (32 expertos, 4 activos por token) significa que aunque el archivo pesa 14GB, solo se activan 3.6B parámetros en cada paso de inferencia. Esto le da un balance único entre calidad de razonamiento y velocidad en CPU.

Configura estas variables de entorno en Windows para optimizar el rendimiento:

```
setx OLLAMA_MAX_LOADED_MODELS 1
setx OLLAMA_NUM_PARALLEL 1
setx OLLAMA_KV_CACHE_TYPE q8_0
setx OLLAMA_KEEP_ALIVE 24h
```

`OLLAMA_KV_CACHE_TYPE=q8_0` reduce a la mitad el uso de memoria del KV cache.

`OLLAMA_KEEP_ALIVE=24h` mantiene el modelo cargado en RAM y evita la recarga de 14GB cada vez que haces una consulta. **Limitar a 1 modelo cargado y 1 solicitud paralela** es esencial en CPU para evitar que el sistema se quede sin memoria.

Conclusión

La ruta más directa es `ollama launch openclaw` — un comando, cero configuración manual. Si necesitas control fino, editar `~/.openclaw/openclaw.json` con los IDs exactos de los modelos (verificados con `ollama list`) elimina el error "Unknown model" sin instalar nada adicional. Lynkr es innecesario y problemático en Windows; si alguna vez necesitas un proxy intermedio, LiteLLM es la alternativa sin fricción que se instala con pip en segundos. La combinación de `gpt-oss:20b` con contexto limitado a 8192 tokens y caché KV en `q8_0` ofrece el mejor equilibrio entre inteligencia y velocidad para un sistema sin GPU con 32GB de RAM.